

Optimizing Master-Slave Broadcast Communication in Multi-Node Network Using RS485 Standard

Md Shakil Tanvir^{*}, Fiaz Ahmed Tonmoy[†], Md Monirul Islam[‡], Mainul Islam[§], Mahamudul Hasan[¶]

^{*}Department of EEE, Ahsanullah University of Science & Technology, Email:shakiltanvir99@gmail.com

[†]Department of BME, Bangladesh University of Engineering and Technology, Email:fiaz.tonmoy@gmail.com

[‡]Department of EEE, Bangladesh University of Engineering and Technology, Email:monirulislam.acad@gmail.com

[§]Department of EEE, Islamic University of Technology, Email:mainulislam@iut-dhaka.edu

[¶]Department of EEE, Daffodil International University, Email:hasan271096@gmail.com

Abstract—This paper presents a comprehensive study on the implementation of master-slave Broadcast Communication over a differential signalling standard (RS-485) that defines the electrical interface and physical layer for connecting electrical devices. The focus is on designing and timing analysis of a communication system involving multiple STM32 microcontrollers arranged in a daisy chain configuration. The master device broadcasts commands to multiple slave devices, which, after receiving the commands, sends feedback to the master as well as other slaves. The system uses Direct Memory Access (DMA) for efficient data transfer, Cyclic Redundancy Check (CRC) for error detection and precise timing control to ensure reliable communication. A key feature of this system is its unique feedback mechanism, where all nodes receive data and execute commands, but only the addressed node provides feedback. Multiple nodes then accept this feedback, ensuring synchronized operation and real-time updates across the network. This study highlights implementing a communication system in a project involving a master-slave configuration, where the master node initiates communication, processes feedback, and adjusts timing. The system has been rigorously tested on a Remotely Operated Vehicle (ROV) with a 6-degree of Freedom (DOF) arm and differential drive, demonstrating its reliability and efficiency in controlling complex robotic movements and feedback mechanisms.

Index Terms—UART, RS-485, DMA, CRC, STM32, DOF, ROV, Slave Id, Multiple nodes

I. INTRODUCTION

In the realm of automation and robotics, particularly those involving complex systems such as multi-degree-of-freedom (DOF) manipulators, drive systems and sensors, efficient communication between multiple control nodes is crucial. A reliable real-time communication system that can handle multiple nodes while providing accurate feedback is vital. This paper introduces a multi-node communication system based on RS485 serial interface and STM32-based microcontrollers. The system is engineered to control a robotic arm with 6 DOF, a differential drive system, cameras, and sensors, all interconnected through a daisy chain network topology. Meanwhile, taking the possibility of increasing numbers of nodes in the system, an efficient and stable addressing mechanism would be necessary for locating a specified node [4]. To address this, the master and slave were assigned unique headers, and each slave was given a unique ID number to distinguish between them. The control system can carry out the control arithmetic

on the master Node and control the slave Node through the bus [12].

II. LITERATURE REVIEW

Serial communication protocols are vital in modern electronics, enabling efficient data transfer between devices across various applications such as embedded systems, industrial automation and the Internet of Things (IoT). Among the numerous protocols, Universal Asynchronous Receiver/Transmitter (UART) protocol and RS485 electrical interface are widely used due to ease of use and robustness.

UARTs are widely used in microcontrollers and embedded systems to facilitate communication between devices such as computers, sensors, and other peripherals. Unlike synchronous communication protocols, UART operates asynchronously, which does not require a shared clock signal between communicating devices [2].

A conventional UART frame consists of a start bit, five to eight data bits, one stop bit, and an optional parity bit for error detection, which can be even or odd [11]. Here, UART communication is implemented by transmitting data in 8N1 format (8-bit data, N - no parity, 1 stop bit) [1] [2].

RS485 is a differential signalling standard, meaning it sends data over twisted pair wires (often referred to as A and B or + and -) [7]. This method helps to cancel out noise and interference, improving signal integrity over long distances. It also employs termination resistance to reduce signal reflection.

RS485 is a crucial protocol in industrial automation, building management systems, and other applications requiring reliable, long-distance communication, particularly in multi-device networks like Modbus RTU and DMX512 [4] [6]. A notable implementation of a flexible RS485-based platform is described in [13], where the authors present a system that allows users to manage network nodes using a graphical programming language. Further advancements in RS485 communication are demonstrated in the work of [14], where the authors successfully realize a multi-master communication system based on RS485. Another study [15] examines the stability of parallel inverters in a master-slave configuration over a CAN bus, focusing on the effects of controller gains and time delays on system performance. In [16], the authors successfully implement Modbus RTU and Modbus TCP

communication protocols using Siemens S7-1200 PLCs to control a batch process. The communication setup allows for remote control and monitoring using various controllers and protocols. While our system currently focuses on a single protocol (UART with RS485), the multi-protocol capability described in [16] could be explored in future work to allow for greater interoperability between different communication systems, further expanding the potential applications of our network.

III. SYSTEM ARCHITECTURE

The system architecture centres around a master-slave configuration using an RS485 standard bus over UART. Each node, equipped with an STM32 microcontroller, can receive commands, execute operations, and provide feedback. The master node orchestrates the communication, ensuring commands are broadcasted and the appropriate node is addressed for feedback.

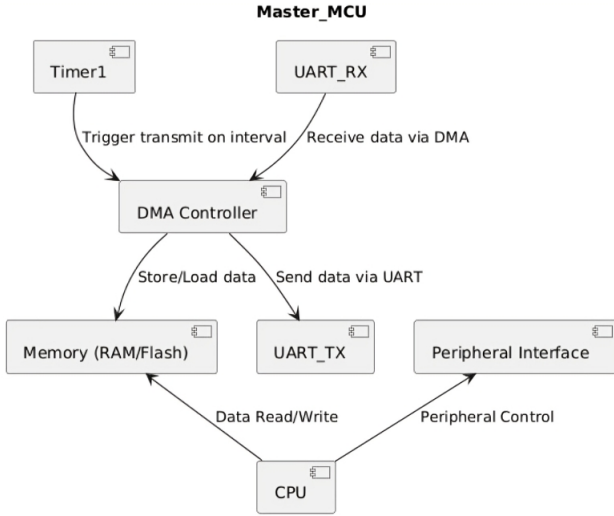


Figure 3.1 Master MCU Architecture

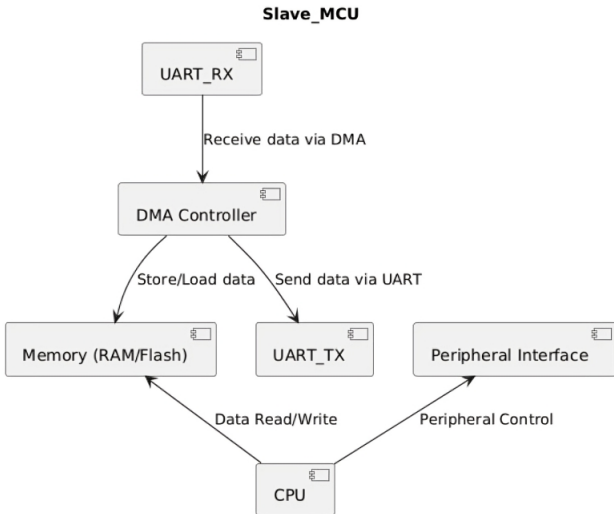


Figure 3.2 Slave MCU Architecture

A. Multi-Node Communication

The communication system enables all nodes to receive data simultaneously and execute the commands. The master node broadcasts commands to all nodes and addresses a specific node to provide feedback. This feedback is then propagated to all other nodes in the network, ensuring a synchronized and consistent state throughout the system.

B. Feedback System

One of the most significant advantages of this system is its unique feedback mechanism. Unlike conventional systems, where feedback is limited to the master node, our system allows all nodes to receive the feedback simultaneously. This capability ensures that all nodes are updated with the latest system status, reducing the risk of desynchronization and improving the system's overall reliability.

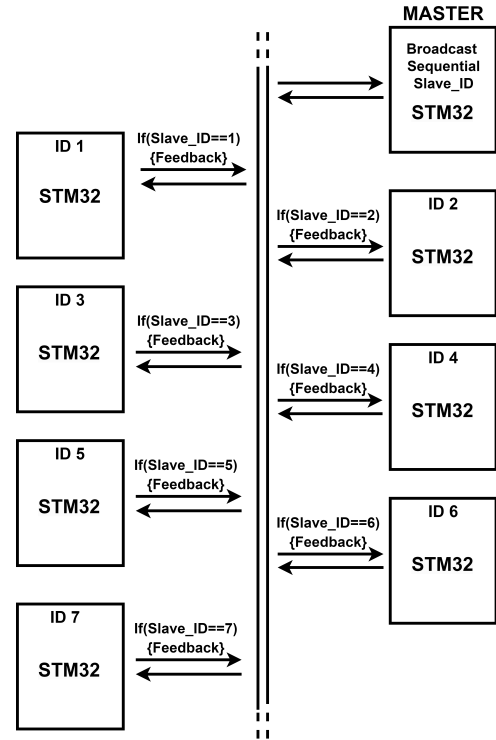


Figure 3.3 Master-Slave Communication Data flow

IV. METHODOLOGY

A. Master-Slave Communication

1) **Data Type:** In the proposed system [Fig. 4.1], the master sends 96 bytes in the communication system, and each slave sends 28 bytes back as feedback. Data size depends on the parameters of the system. The baud rate maintained is 115200 bps. Here, data contains header bytes for master and slave identification, slave ID bytes for node identification, other bytes containing information about different sensors and actuators, and the last bytes are related to CRC (Cyclic Redundancy Check).

Header	Slave ID	Data Size	Data	CRC
--------	----------	-----------	------	-----

Figure 4.1 Data type of the proposed solution

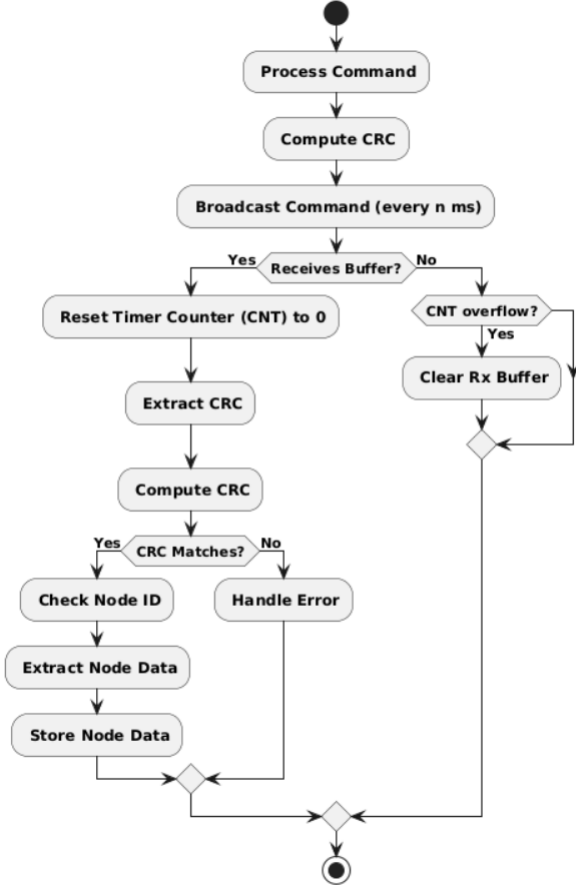


Figure 4.2 Master communication algorithm

2) **Master Implementation:** This flowchart [Fig. 4.2] demonstrates the process of the master broadcasting command via the RS485 bus to all slaves and the master receiving feedback via the RS485 bus from all slaves. Here, n represents each master-slave communication cycle time.

3) **Slave ID Implementation:** In a master-slave communication configuration [Fig 4.3], each slave device is given a unique ID called Slave ID, which enables the master to differentiate between several slaves on the same communication channel.

Using the communication bus, the master broadcasts commands and the slave ID. To guarantee that the message is routed appropriately, only the slave who has the corresponding ID will reply to the master's request. Each slave listens on the communication bus for its ID. Each slave reads the given command, interprets it, carries it out, and looks through the packet for its ID. Feedback is transmitted if the ID matches. Address pins (for hardware-based identification) or software configuration can be used to implement slave IDs. The address (Slave ID) is usually included in the protocol specification for protocols such as I2C, which makes implementation easier. Dynamic addressing or arbitration might be required in increasingly complicated systems to manage several devices efficiently.

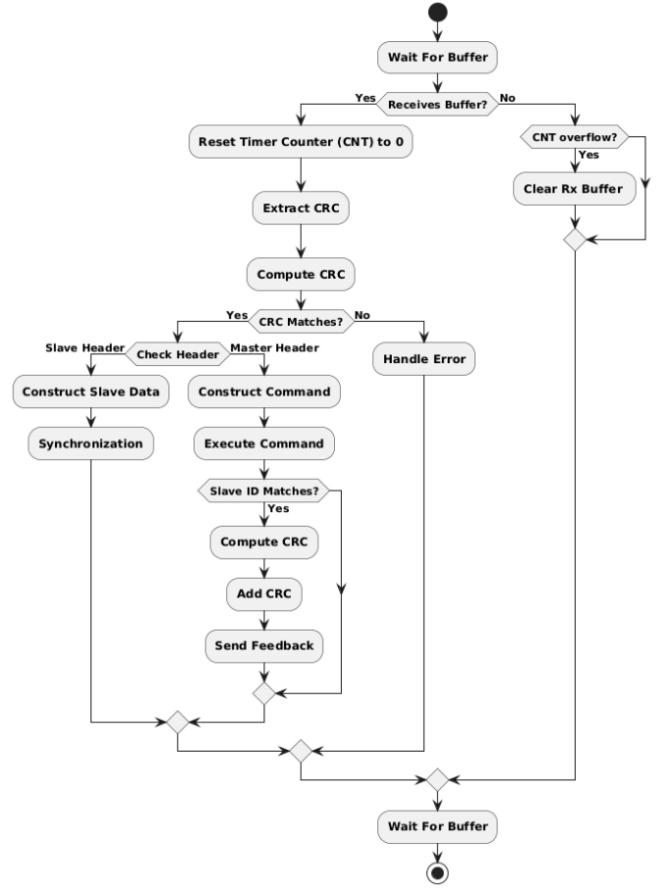


Figure 4.3 Slave communication algorithm

4) **Slave Implementation:** This flowchart [Fig. 4.3] demonstrates the process of each slave receiving a command via the RS485 bus from the master, as well as feedback from other slaves and broadcasting feedback to the master and other slaves.

5) **DMA Implentation:** Peripherals (such as serial ports) can access system memory directly due to a technology called direct memory access (DMA), which spares the CPU from this task.

6) **CRC Implementation:** The Cyclic Redundancy Check (CRC) technique is a means of identifying data transmission issues. CRC is essential for maintaining data integrity in master-slave communication, especially when communicating over long distances or when interference may occur [5].

B. Timing

The transmission time (TX) and reception (RX) must be accurately synchronized to guarantee proper data exchange. Usually, the master starts the conversation by sending data across the RS485 line. The baud rate at which data bits are transferred controls the timing of this transmission. To provide the slave enough time to process the data and respond, the master must ensure a sufficient interval between each transfer: the slave and the master must be able to receive data at the same baud rate. The slave watches the RX line for any data

from the master. The slave receives the data, processes it, and gets ready to respond. Full-duplex communication enables simultaneous TX and RX [3]

C. Slave Feedback to Master

Data that have been received from the slave or data that need to be delivered to the slave are kept in the master's buffer. When data becomes available, the master reads them from the buffer and processes them immediately. Similar to this, the slave's buffer holds data that is to be transmitted back to the master, or that is received from it. As soon as data are received, the slave processes them to prevent the buffer from overflowing. A data erase timer can remove old or unprocessed data from the buffer after a predetermined time.

D. Slave-Slave Communication

Master uses UART-based data transmission over the RS485 interface. Any slave connected to the bus can listen to it and generate feedback in the same way when addressed. Secondly, each slave, when addressed, uses UART-based data transmission over the RS485 interface. The master, as well as any slave connected to the bus, can listen to it. So, if there are N number of slaves, then after N cycles of command broadcast by master: (i) each slave has received command from master N times, (ii) master has received feedback from each slave 1 time, (iii) each slave has received feedback from every other slave 1 time.

V. EXPERIMENTAL RESULTS

A. Circuit Description

Below is a sample circuit description [Fig. 5.1], demo comprises 4 STM32G431KB where three are slaves, and one is the master. All mcus are connected in a Daisy fashion, where the master is connected in the first chain and the slaves in the rest of the chain as nodes. The master provides necessary commands to the slaves, and the slaves, after performing the command, give feedback back to the master

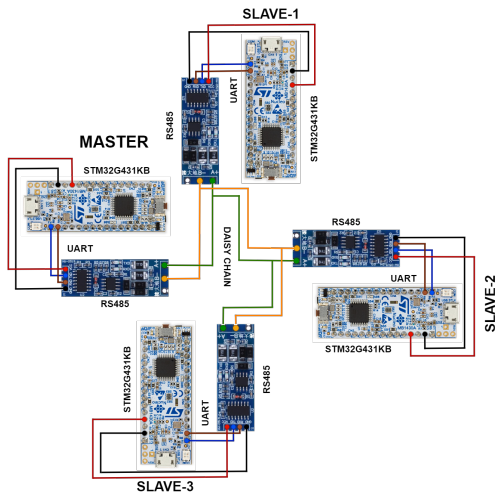


Figure 5.1 Hardware Wiring Diagram

The following is the robot circuit description [Fig. 5.2]. The robot comprises 11 STM32G431GB as slaves and 1

STM32F446RE as the master. All MCUs are connected in the Daisy fashion [8] [9]. The master provides necessary commands to the slaves, and the slaves, after performing the command, give feedback to the master.



Figure 5.2 Robot Hardware Setup

B. Time Delay Calculation

Let n be the number of slaves (microcontrollers) in the communication system, b be the baud rate (number of bits per second), N_B be the number of bytes, and N_b be the number of bits. If we consider the transmission time for the master command to reach a slave is T_1 , the time taken for command translation command realization, then the slave generating feedback is T_2 , the time taken for the transmission of the feedback packet from the slave using UART is T_3 , finally the time for the master to process feedback data and generate command data is T_4 , then the consecutive master-slave communication delay, which is t . To provide the slave enough time to process the data and respond, the master must ensure a sufficient interval between each transfer. So,

$$(T_1 + T_2 + T_3 + T_4) < t \quad (1)$$

To determine the time taken for n slaves to perform master-slave communication once, the n slave command feedback cycle time T will be

$$T = n \times t \quad (2)$$

For simplicity, we consider the highest processing time in the most computationally expensive slave node to be the universal processing time for each slave node. The bottleneck in processing time will be the maximum time it takes to process data in any slave node. It is expressed as:

$$T_2 \equiv T_{process} \quad (3)$$

If we consider the number of command packets to be N_c and the number of feedback packets to be N_f , where both are expressed in bytes, then,

$$T_1 = (N_c \times N)/b \quad (4)$$

$$T_3 = (N_f \times N)/b \quad (5)$$

Where T_1 is the command UART period, T_3 is the feedback UART period, and N is the number of start bits, data bits, stop bits, and parity bits.

Considering everything, the delay estimation in master-to-each-slave communication is a modification of equation (7), that is,

$$t > N \times (N_c + N_f)/b + T_{process} + T_4 \quad (6)$$

For n number of slaves, the overall delay observed from equation (12),

$$T = n \times t \quad (7)$$

C. Observation

The system was successfully tested in a real-world scenario, where all 11 slave nodes operated seamlessly in coordination. The feedback mechanism proved effective, ensuring that all nodes received real-time updates. This resulted in a coherent and synchronized robot operation, indicating the system's reliability in complex robotic applications.

The robot circuit setup includes the practical setup [Fig. 5.2]. A logic analyzer was used to analyze the timing behaviour using the setup of [Fig. 5.2]. From [Fig. 5.3] channel 1 represents the master sending data to all slaves. The channel 2 represents slave feedback data. The master-slave communication is ideally 20 ms per cycle. Every slave gets a chance to send feedback when the master requests feedback. Since there are 11 slaves, each slave should give feedback once every 11×20 ms or 220 ms.

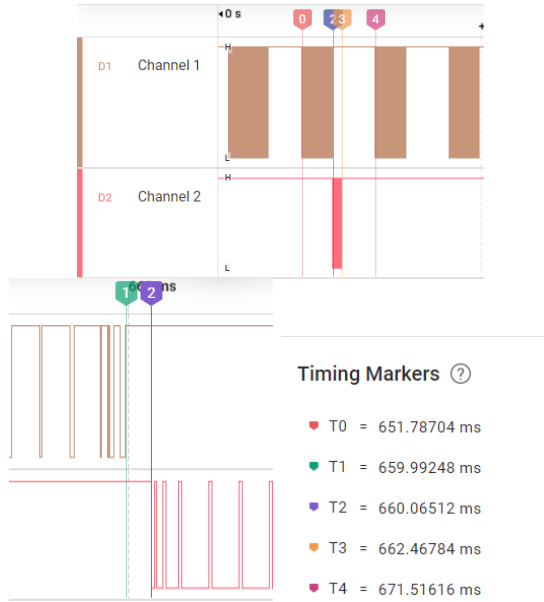


Figure 5.3 Timestamps from logic analyzer

From figure 5.3 we can write T_1, T_2, T_3, T_4 as:

$$T_1 = (T1 - T0) = 8.20544ms \quad (8)$$

$$T_2 = (T2 - T1) = 72.64us \quad (9)$$

$$T_3 = (T3 - T2) = 2.40272ms \quad (10)$$

$$T_4 < (T4 - T3) = 9.04832ms \quad (11)$$

$$t = (T4 - T0) = 19.72912ms \quad (12)$$

Now, if we sum equations 8 to 10 and asymmetry 11, we get,

$$(T_1 + T_2 + T_3 + T_4) < T4 - T0 = 19.72912ms \quad (13)$$

For our system,

$$N = 8N1 = 9bits/byte \quad (14)$$

$$N_c = 96bytes \quad (15)$$

$$N_f = 28bytes \quad (16)$$

$$b = 115200bits/sec \quad (17)$$

After calculating, we get

$$T_1 = 7.5ms \quad (18)$$

$$T_3 = 2.1875ms \quad (19)$$

Here, T_2 and T_4 are internal processing times dependent on the MCU. From observation,

$$T_2 = 72.64us \quad (20)$$

They are comparable since T_2 and T_4 are both internal processing times. So,

$$T_2 \equiv T_4 = 72.64us \quad (21)$$

If we calculate the minimum time required for successful communication,

$$(T_1 + T_2 + T_3 + T_4) = 9.83278ms < t = 19.72912ms \quad (22)$$

Here, the minimum time required for one successful command and feedback communication cycle is 9.83278 ms. If the periodic broadcast window is less than that, there will be data collision between command and feedback in the RS485 wires. The periodic broadcast window is 19.72912ms, which is much greater. The value T_4 we assumed to be 72.64us will not be of the order of milliseconds, whereas the idle time of communication is $19.72912ms - 9.83278ms = 9.89634$ ms. So, there is no chance of data collision.

The idle time depends on the baud rate of the system, and if the baud rate is reduced, then the idle time will decrease and vice versa.

The overall system is shown in figure [Fig. 5.4]. Here, we can see that each slave sends feedback every 56.57ms, which is close to our theoretical calculations. Here, every cycle is performed in 20ms.



Figure 5.4 Master - Slave broadcast at 20ms (only 3 slaves are shown)

The figure shows the complete communication cycle for master-slave communication. Here, channel 4 represents the master command and channels 1 to 3 represent slave feedback from different slaves. Channel 5 shows feedback data that the master receives from all the slaves.

VI. CONCLUSION

This system can be used where low latency control is required and an easy communication system is preferred, such as open-source robotic platforms. Controlling one slave per command packet induces huge latency when the number of slaves increases. However, controlling all slaves with one large command packet and taking feedback one slave at a time with just a single bus can significantly reduce command latency, which is crucial for robotic systems such as differential drive systems for ROV. Low-latency control is an essential requirement for motion control technologies. It also has all the advantages of the RS485 interface [6]. All other nodes must have their transmission port on standby when one node transmits. All nodes cannot give feedback simultaneously. So feedback is not received from a specific slave immediately after a command is issued, but rather when the master commands for feedback by addressing a specific slave through its id.

A. Future Aspects and improvements

The complete system uses an MCU timer and communication interrupts to achieve a real-time response. It can be further improved using the Real-Time Operating System (RTOS) [10].

B. Acknowledgement

Special thanks to Spectrum Engineering Consortium (Pvt.) Ltd for funding and technical assistance.

REFERENCES

- [1] Nanda, U. and Pattnaik, S.K., 2016, January. Universal asynchronous receiver and transmitter (UART). In 2016 3rd international conference on advanced computing and communication systems (ICACCS) (Vol. 1, pp. 1-5). IEEE.
- [2] Gupta, A. and Gupta, A., 2019. UART communication. The IoT hacker's handbook: a practical guide to hacking the Internet of things, pp.59-80.
- [3] Gupta, A.K., Raman, A., Kumar, N. and Ranjan, R., 2020, February. Design and implementation of high-speed universal asynchronous receiver and transmitter (UART). In 2020 7th International Conference on Signal Processing and Integrated Networks (SPIN) (pp. 295-300). IEEE.
- [4] Min, D., Xiao, Z., Sheng, B. and Shiya, G., 2012, May. Design and implementation of the multi-channel RS485 IOT gateway. In 2012 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER) (pp. 366-370). IEEE.
- [5] Jin, Y. and Yang, J., 2011, August. Design an intelligent environment control system for greenhouse based on rs485 bus. In 2011 second international conference on digital manufacturing & automation (pp. 361-364). IEEE.
- [6] Li, L., Lu, Q. and Yin, G., 2017. Design of bus protocol intelligent initiation system based on RS485. In MATEC Web of Conferences (Vol. 100, p. 03003). EDP Sciences.
- [7] Cheng-ying, Y. and Yong, C., 2019, June. Research on multi-master communication system based on RS485 bus. In Journal of physics: conference series (Vol. 1237, No. 4, p. 042078). IOP Publishing.
- [8] Amin, I., Brushed, D.C., Motors, S. and Lockridge, J., 2021. Daisy Chain Implementation for Serial Peripheral Interface.
- [9] Sudev, B., Kinghorn, I., Gu, D. and Gower, D., Modbus-A: Automated Slave ID Allocation Enabling Architecture for Modbus Devices on RS485/232.
- [10] Hambarde, P., Varma, R. and Jha, S., 2014, January. The survey of real time operating system: RTOS. In 2014 International Conference on Electronic Systems, Signal Processing and Computing Technologies (pp. 34-39). IEEE.
- [11] Dhanadraye, A.D. and Thorat, S.S., 2014. A review on implementation of UART using different techniques. International Journal of Computer Science and Information Technologies, 5(1), pp.394-396.
- [12] Yu, S., Chen, W., Li, L. and Qin, J., 2006, June. Development of ARM-based embedded system for robot applications. In 2006 IEEE Conference on Robotics, Automation and Mechatronics (pp. 1-6). IEEE.
- [13] Hung, P.D., Van Chin, V., Chinh, N.T. and Tung, T.D., 2020. A Flexible Platform for Industrial Applications Based on RS485 Networks. J. Commun., 15(3), pp.245-255.
- [14] Cheng-ying, Y. and Yong, C., 2019, June. Research on multi-master communication system based on RS485 bus. In Journal of physics: conference series (Vol. 1237, No. 4, p. 042078). IOP Publishing.
- [15] Alfergani, A., Khalil, A., Rajab, Z., Zuheir, M., Asheibi, A., Khan, S., Aboadla, E.H.E., Azna, K.A.B. and Tohtayong, M., 2017, October. Control of master-slave microgrid based on CAN bus. In 2017 IEEE Jordan Conference on Applied Electrical Engineering and Computing Technologies (AEECT) (pp. 1-6). IEEE.
- [16] Tamboli, S., Rawale, M., Thoraiet, R. and Agashe, S., 2015, May. Implementation of Modbus RTU and Modbus TCP communication using Siemens S7-1200 PLC for batch process. In 2015 international conference on smart technologies and management for computing, communication, controls, energy and materials (ICSTM) (pp. 258-263). IEEE.